

NATIONAL UNIVERSITY OF SINGAPORE
SCHOOL OF COMPUTING
MIDTERM TEST
AY2025/26 Semester 1
CS2100 – COMPUTER ORGANIZATION
DURATION OF ASSESSMENT: 1.5 HOURS

DRAFT ANSWERS

We are providing the answers for this midterm to further your understanding of the subject. We hope you carefully review it and catch the part where you had an issue, improving your knowledge of the subject matter.

Part A: Multiple Choice Questions (25 marks)

25 questions, 1 mark each. Each question has exactly one correct answer. On the Answer Sheet, shade the circle for your chosen option firmly with a dark pencil; do not tick, cross, or write. Ensure other options remain unmarked. MCQs are machine-graded; unclear or tainted marks may not be captured. We are using machine recognition to perform grading.

Question 1

Which of the following is the 8-bit binary signed magnitude sign extension of the 4-bit binary signed magnitude number 1010_{sm} ?

- A. 1000010_{sm}
- B. 10001010_{sm}
- C. 11110010_{sm}
- D. 11111010_{sm}
- E. None of the above

The value in 4-bit signed magnitude is -2_{10} .
So is A.

Question 2

Which of the following decimal values is not representable in a 9-bit binary signed magnitude representation?

- A. +128
- B. -1
- C. +290
- D. -254
- E. None of the above

Take away the MSB for sign bit, we have 8-bits for values that can range from $0x00$ to $0xFF$, i.e., 0 to 255. So the most positive representable would be +255 and the most negative representable would be -255.

Question 3

Suppose Mary has a 32-bit two's complement integer X that she wishes to multiply by 5. Which one of the following sequences of steps would **not** work? Assume that all intermediate results fit in 32 bits with no overflow.

- A. Shift X left by 1 bit to get Y , then shift Y left by 1 bit to get Z and finally add X to Z to get $X * 5$.
- B. Shift X left by 2 bit to get Y , then add X to Y to get $X * 5$.
- C. Shift X left by 3 bit to get Y , then subtract X from Y to get $X * 5$.
- D. Shift X left by 1 bit to get Y , and shift X left by 1 bit to get Z , and finally do $X + Y + Z$ to get $X * 5$.
- E. None of the above

Question 4

Which one of the following correctly states the bases involved in the following arithmetic expression involving unsigned values of their respective bases?

$$4090_X - 1520_Y = 3206_Z$$

- A. $X = 10$, $Y = 7$ and $Z = 7$
- B. $X = 15$, $Y = 13$ and $Z = 8$
- C. $X = 12$, $Y = 11$ and $Z = 9$
- D. $X = 11$, $Y = 13$ and $Z = 9$**
- E. None of the above

A	$4090_{10} = 4090$ $1520_7 = 602$ $3206_7 = 1133$ $4090 - 602 = 3488 (!= 1133)$
B	$4090_{15} = 13635$ $1520_{13} = 3068$ $3206_8 = 1670$ $13635 - 3068 = 10567 (!= 1670)$
C	$4090_{12} = 7020$ $1520_{11} = 1958$ $3206_9 = 2355$ $7020 - 1958 = 5062 (!= 2355)$
D	$4090_{11} = 5423$ $1520_{13} = 3068$ $3206_9 = 2355$ $5423 - 3068 = 2355$

Hence the answer is D.

Question 5

Which of the following is the 8-hexadecimal-digit hexadecimal string that represents the floating point value -1.0 (decimal) in the IEEE Standard 754 single precision (32-bit) floating point?

- A. **0xbf800000**
- B. 0xbe800000
- C. 0x3f800000
- D. 0x3e800000
- E. None of the above

Sign bit is 1.

Exponent is 0. After adding bias, it is 127, or 0x7f.

Mantissa is 0.

Putting it all together, we get

1|011 1111 1|000 0000 0000 0000 0000 0000₂

which is A.

Question 6

Bob wrote the following code while studying the IEEE Standard 754 single precision (32-bit) floating point:

```
#include <stdio.h>

union {
    unsigned int i;
    float f;
} u;

void plusConvert(int i)
{
    u.f = (float)i; // Convert to IEEE single precision
    printf("1. i = 0x%x (%d): 0x%x %e\n", i, i, u.i, u.f);

    i++;

    u.f = (float)i; // Convert to IEEE single precision
    printf("2. i = 0x%x (%d): 0x%x %e\n\n", i, i, u.i, u.f);
}

int main(int argc, char *argv[])
{
    int X = <some value>;

    plusConvert(X);
}
```

The function `plusConvert()` takes a 32-bit two's complement integer as an argument, X , and converts it into a IEEE Standard 754 single precision (32-bit) floating point number using C's type casting and "round to zero" (which simply discards any extra bits). He then use a union to print out the exact bits of the converted floating point value. Let's call this $f(X)$. To his surprise, he found that if he does the same with $X+1$, (let's call this $f(X+1)$ – note that the 1 is added before the conversion to floating point), he found that for *some* values of X , $f(X)$ is equal to $f(X+1)$ in that the binary representations are identical, even though $X \neq X+1$ (as 32-bit two's complement integers). Which of the following integer would be one such possible X ? (Note: the numbers below are all in hexadecimals.)

- A. $X = 0x25a2d87f$
- B. $X = 0x0120101c$
- C. $X = 0x0388922f$
- D. $X = 0x00a66d41$
- E. None of the above

This question is a bit more involved. The key insight is that in IEEE 754 32-bit binary floating point format, the mantissa (excluding the implicit 1) is only 23 bits. This means that it cannot represent all the 32-bit integers. In particular, it depends on the spread of 1's and not just the magnitude. So, for example, the (positive) integer $0110\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_2$ (32 bits) can still be represented exactly as 1.1×2^{30} . But in trying to represent say $0110\ 0000\ 0000\ 0000\ 0000\ 0000\ 1111_2$ (32 bits), we will be forced to drop the last 4 1's simply because we do not have 32 bits for our mantissa, and that will result in the phenomenon described in the question.

So if we expand each of the numbers, we find that:

$$A = 0x25a2d87f = 0010\ 0101\ 1010\ 0010\ 1101\ 1000\ 0111_2$$

$$A+1 = 0x25a2d880 = 0010\ 0101\ 1010\ 0010\ 1101\ 1000\ 1000_2$$

$$B = 0x0120101c = 0000\ 0001\ 0010\ 0000\ 0001\ 0000\ 0001\ 1100_2$$

$$B+1 = 0x0120101d = 0000\ 0001\ 0010\ 0000\ 0001\ 0000\ 0001\ 1101_2$$

$$C = 0x0388922f = 0000\ 0011\ 1000\ 1000\ 1001\ 0010\ 0010\ 1111_2$$

$$C+1 = 0x03889230 = 0000\ 0011\ 1000\ 1000\ 1001\ 0010\ 0011\ 0000_2$$

$$D = 0x00a66d41 = 0000\ 0000\ 1010\ 0110\ 0110\ 1101\ 0100\ 0001_2$$

$$D+1 = 0x00a66d42 = 0000\ 0000\ 1010\ 0110\ 0110\ 1101\ 0100\ 0010_2$$

The underlined bits are the 23 bits that make it into the mantissa, the bit in bold being the least significant one of these. You can see that the two values for B and B+1 results in the same 23 bits in the mantissa.

Question 7

Which of the following is a valid C initialization statement (i.e., no compilation warning or error and code will run accordingly)?

- | | |
|----|--|
| A. | <pre>struct { int x; char s[3]; int y; } stu = {, "abc"};</pre> |
| B. | <pre>struct { int x; char s[3]; int y; } stu = {1, "abcd"};</pre> |
| C. | <pre>struct { int x; char s[3]; int y; } stu = {1, "abc"};</pre> |
| D. | <pre>struct { int x; char s[3]; int y; } stu = {1, "abc", 2, 3};</pre> |
| E. | None of the above |

Question 8

Which of the following **is a valid** C **if** statement (i.e., no compilation warning or error, and the code will run accordingly), assuming the body of the loop (indicated as "...") is all good? We will assume that any variable referred to here is properly defined.

- A. `if () {...}`
- B. `if (;) {...}`
- C. `if (i;) {...}`
- D. `if (i?0:1) {...}`
- E. None of the above

Question 9

Consider the following C snippet:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    for (int i=0; i<10; i++)
        printf("%d ", ++i);
}
```

What output would be printed out at the terminal after compiling and executing this C loop?

- A. 1 3 5 7 9
- B. 1 2 4 8 9
- C. 0 2 4 6 8
- D. 1 2 3 4 5 6 7 8 9
- E. None of the above

Question 10

Consider the following C program:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i;

    i = 5;
    do {
        printf("%d ", i--);
    } while (--i && i>0);
}
```

What output would be printed out at the terminal after compiling and executing this program?

- A. 4 2 0
- B. 5 3 1
- C. 4 3 2 1 0
- D. 5 4 3 2 1
- E. None of the above

Question 11

Consider the following C code:

```
#include <stdio.h>

int A[4] = {1, 2, 3, 4};

int main(int argc, char *argv[])
{
    int *p[2], **q;

    p[0] = &(A[0]);
    p[1] = &(A[2]);
    q = p;

    *q = &(A[1]);
    **q++ = 100;

    printf("%d %d %d %d\n",
           A[0], A[1], A[2], A[3]);
}
```

What is the output printed to the terminal after compiling and executing this program?

- A. 1 2 3 4
- B. 1 2 3 5
- C. 1 100 3 4
- D. 1 2 3 100
- E. None of the above

Question 12

Assuming the pointers and integers are all 32-bits in length, and the array **A** in Question 11 is stored at location **0x1000** while the array **p** is in location **0x5000**. What would be the value of ***q** (note *not* **q** itself) just before the **printf()** statement in the above code?

- A. 0x1004
- B. 0x1008
- C. 0x5004
- D. 0x5008
- E. None of the above

q is a pointer to a pointer. It's initial value is **0x5000** (address of **p**). In "****q++**", **q** is incremented. So it will point to the second element of **p**, which in turn points to **A[2]** or **0x1008** due to the assignment of **p[1] = &(A[2]);**.

Question 13

What is the hexadecimal encoding for the following instruction:

`lw $v1, -100($t7)`

- A. `0x8c6fff9c`
- B. `0x8c6fff9d`
- C. `0x8de3ff9b`
- D. `0x8de3ff9c`
- E. None of the above

Question 14

The following is the hexadecimal encoding of a branch instruction in MIPS:

`0x12c3fbad`

Assume that this instruction is at PC = `0x5204`. What is the target PC if the branch is taken?

- A. `0x40b0`
- B. `0x40b8`
- C. `0x40bc`
- D. `0x53b8`
- E. None of the above

`0xfbad` extended to 32-bit gives the constant -1107_{10} . Multiply this by 4, we get -4428_{10} (`-0x114c`). Next we add this to PC+4, and we get `0x40bc`.

Question 15

The “move” instruction is a pseudo-instruction. It moves the content of one register to another. Which of the following cannot be used to implement the following pseudo instruction that moves the content of register `$r2` to register `$r1` because for some inputs, it produces the wrong results?

`move $r1, $r2`

- A. `addi $r1, $r2, 0`
- B. `andi $r1, $r2, 0xff`
- C. `or $r1, $r2, $zero`
- D. `sll $r1, $r2, 0`
- E. None of the above

This is because `andi` will do zero extension of the immediate. The upper half of `$r1` will be zero'ed out.

Question 16

Consider the following MIPS instruction given in hexadecimal:

0x080000ad

Assuming that the PC of this instruction is **0x1014**, which of the following instructions would be equivalent to this instruction in hexadecimal above? All the numbers below are in decimal.

- A. **beq \$v0, \$v0, -857**
- B. **beq \$v0, \$zero, -1024**
- C. **bne \$t0, \$zero, 173**
- D. **bne \$zero, \$zero, -3428**
- E. None of the above

Question 17

Which sequence(s) of instruction(s) correctly sets bit 9, 12, and bit 15 of **\$t0** to 1, with the other bits set to 0?

- | | |
|---|--|
| (i) ori \$t1, \$zero, 0x49
sll \$t1, \$t1, 9
or \$t0, \$t0, \$t1 | (ii) ori \$t0, \$zero, 0xff
addi \$t0, \$t0, 0x9200 |
| (iii) ori \$t0, \$zero, 0x92 | (iv) lui \$t0, 0x124
srl \$t0, \$t0, 9
add \$t0, \$t0, \$t0 |

- A. Only (i) and (ii).
- B. Only (ii) and (iii).
- C. Only (i), and (iv).
- D. Only (iv).
- E. **None of the above.**

The problem with (i) is while the bit pattern is correct in **\$t1** (**0x9200**), the value currently in **\$t0** is unknown and hence cannot reliably produce the result we want.

(ii) will produce **0xffff92ff** due to the sign extension of **0x9200** in the **addi**.

(iii) uses the wrong constant – it should be **0x9200**.

(iv) is totally off, producing **0x12400**.

Hence, “none of the above”.

Question 18

A sender transmits a 32-bit word composed of 31 data bits followed by 1 odd parity bit. The parity is computed over all 31 data bits. The sender is big-endian and the receiver is little-endian. Assuming no transmission errors, which 32-bit data chunks, represented in hexadecimal, will be accepted without a parity error by the receiver?

- A. **0x752d6298**
- B. **0x1ab8407a**
- C. **0xb3fdf0cb**
- D. **All of the above**
- E. None of the above

Option A has 15 ones.

Option B has 13 ones.

Option C has 21 ones.

Hence all satisfies odd parity.

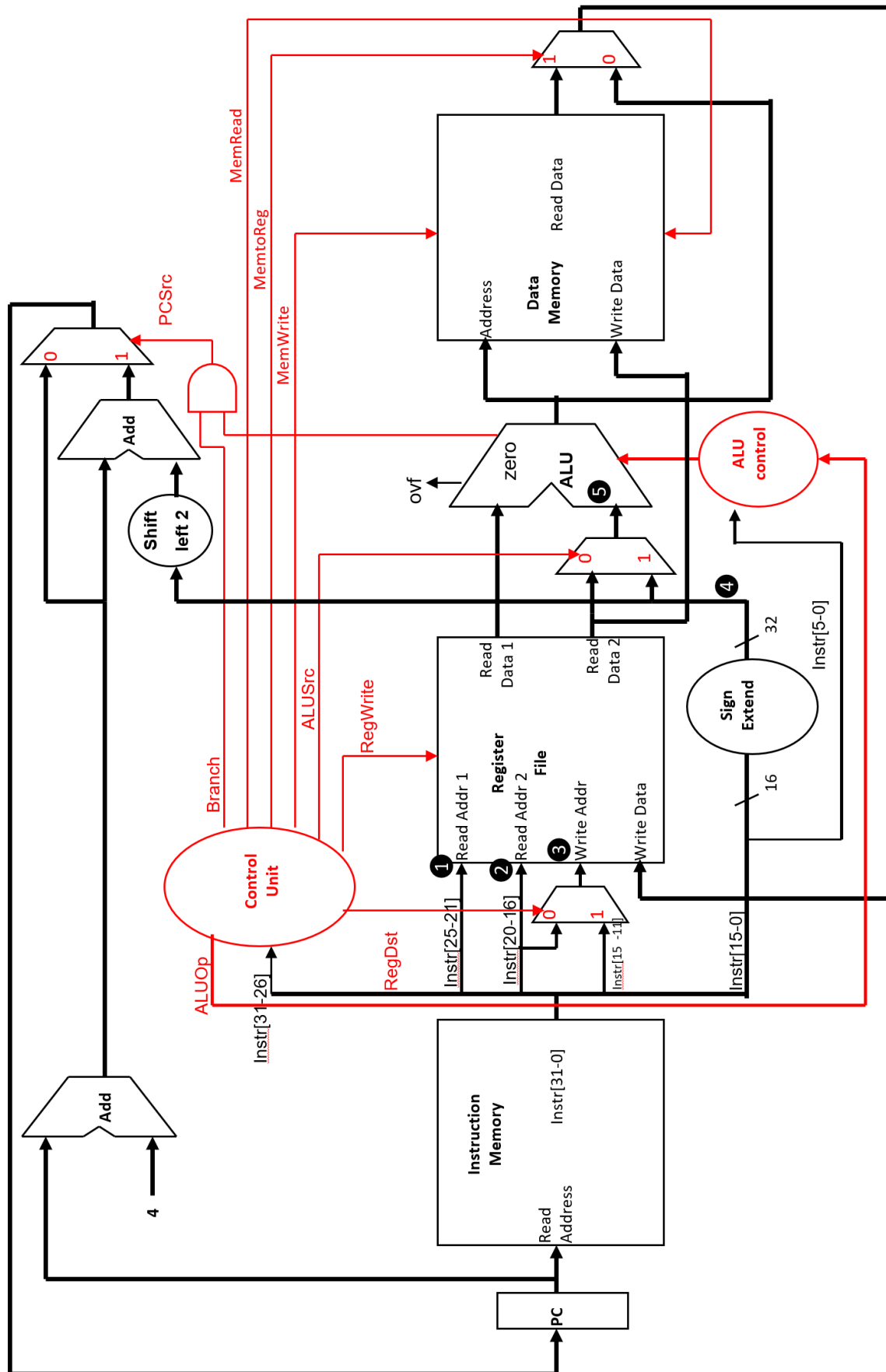
Question 19

What would be the main drawback of an aggressively designed expanding opcode scheme?

- A. **The instruction decoding logic would be complex.**
- B. Difficult for users to use.
- C. Cannot do memory addressing.
- D. All of the above.
- E. None of the above

Question 20-25 shall use the diagram in the following page and these assumptions:

- The instruction being executed is: **add \$t4, \$v0, \$s1**
- The contents of the registers involved are:
 - **\$t4 = 0x100**
 - **\$v0 = 0x200**
 - **\$s1 = 0x300**



Question 20

What is the value at the position marked ① in the diagram? All values below are in decimal.

- A. 2
- B. 12
- C. 17
- D. 20
- E. None of the above

rs is \$v0 or register 2.

Question 21

What is the value at the position marked ② in the diagram? All values below are in decimal.

- A. 2
- B. 12
- C. 17
- D. 20
- E. None of the above

rt is \$s1 or register 17.

Question 22

What is the value at the position marked ③ in the diagram? All values below are in decimal.

- A. 2
- B. 12
- C. 17
- D. 20
- E. None of the above

rd is \$t4 or register 12.

Question 23

What is the value at the position marked ④ in the diagram? All values below are in hexadecimal.

- A. 0x00000000
- B. 0x00006020
- C. 0xFFFF6020
- D. 0xFFFFFFFF
- E. None of the above

The encoding for the instruction is **0x00516020**. The lower 16 bit is fed into the sign extension unit and will produce **0x00006020** – which is then ignored by the ALU because ALUSrc will choose the second register (\$s1) read from the register file instead.

Question 24

What is the value at the position marked ⑤ in the diagram? All values below are in hexadecimal.

- A. 0x00000000
- B. 0x00000100
- C. 0x00000200
- D. 0x00000300
- E. None of the above

The value of rt goes into the ALU as its second input.

Question 25

Which of the following correctly assigns the control signals?

- A. RegDst = 0, RegWrite = 0, ALUSrc = 0, MemRead = 0, MemWrite = 0, MemtoReg = 0
- B. RegDst = 1, RegWrite = 1, ALUSrc = 0, MemRead = 0, MemWrite = 0, MemtoReg = 0
- C. RegDst = 0, RegWrite = 1, ALUSrc = 1, MemRead = 0, MemWrite = 0, MemtoReg = 0
- D. RegDst = 0, RegWrite = 1, ALUSrc = 0, MemRead = 0, MemWrite = 0, MemtoReg = 1
- E. None of the above

We need to use rd as the write register. Hence RegDst = 1, also RegWrite = 1.

ALUSrc = 0 so that we feed the second register file output into the ALU second operand instead of the signed extended immediate.

MemRead = MemWrite = 0 since there is no memory operation.

MemtoReg = 0 so that the ALU output, not memory output, goes back to the register file for writing.

Part B: Short questions (15 marks)

You must answer in the space provided. Any writing outside the answer box will be disregarded.

Question 26 (5 marks)

As mentioned in Assignment 1, MIPS has a real instruction

sllv \$rx, \$ry, \$rz

that performs a logical left shift of **\$ry** by the amount in the lowest 5 bits of **\$rz** and writes the result to **\$rx**. Suppose **sllv** is unavailable and must be implemented as an assembler pseudo-instruction. Give the instruction sequence an assembler could emit to implement this instruction.

Constraints:

- Use only instructions from the Core Instruction Set of the MIPS Reference Data.
- Only the lowest 5 bits of \$rz determine the shift amount (0–31); higher bits in \$rz are ignored.
- The contents of \$ry and \$rz must be preserved throughout the execution.
- Use \$at (\$1) as the temporary register and avoid using other registers as temporaries if possible.

Answer for Question 26:

```

andi $at, $rx, 0x1F    // Need the least 5 bits of $rz; hex 0x1F of 3110
addi $rx, $ry, 0      // Copy $ry into $rx
LOOP:
beq $at, $zero, OUT    // Are we done?
sll $rx, $rx, 1        // Shift by 1 position
addi $at, $at, -1
j LOOP                // Loop
OUT:

```


Question 27 (4 marks)

Suppose there is a machine that has 64 registers, and three types of instructions as follows:

Class A instructions: these are 16-bit instructions that take **two** register operands.

Class B instructions: these are 32-bit instructions that take **three** register operands as well as a 6-bit shift amount.

Class C instructions: these are 32-bit instructions that take **two** register operands and a 14-bit immediate operand.

What is the maximum and minimum number of instructions that this machine can have, assuming all classes exist, and the encoding space for opcode is completely utilized?

Answer for Question 27.

Maximum number of instructions: _____

1 Class A instruction (say binary opcode 0000_2), 1 Class C instruction (binary opcode say 000100_2), then Class B instructions can go from 00010100_2 (20_{10}) to 11111111_2 (255_{10}), or $255 - 20 + 1 = 236$ instructions. So the maximum is $236 + 1 + 1 = 238$.

Minimum number of instructions: _____

4 Class B instruction (say binary opcode 00000000_2 to 00000011_2). 3 Class C instruction (000001_2 to 000011_2) and 15 Class A, i.e., 0001_2 to 1111_2 . So the minimum is $4 + 3 + 15 = 22$.

Question 28 (6 marks)

Referring to the diagram for Question 20 above, and the following table:

Inst-Mem	Adder	MUX	ALU	Reg-File	Data-Mem	Control / ALU-control	Left-shift/ Sign-Extend/ AND
400ps	100ps	30ps	120ps	200ps	350ps	100ps	20ps

Note that “Adder” and “MUX” refers to any adder (except the ALU) or mux in the diagram. Give the estimated latency of an “**addi**” instruction. In your answer, you should identify which column of the above table is used for the computation, and which are not, as well as whether it is on the critical path of the instruction.

Answer for Question 28:

Inst-Mem: (Tick one) ☐ On the critical path ☐ Not on the critical path

On the critical path.

Adder: (Tick one) ☐ On the critical path ☐ Not on the critical path

Not on the critical path.

MUX: (Tick one) ☐ On the critical path ☐ Not on the critical path

On the critical path - twice.

Reg-File: (Tick one) ☐ On the critical path ☐ Not on the critical path

On the critical path - twice.

Data-Mem: (Tick one) ☐ On the critical path ☐ Not on the critical path

Not on the critical path.

Control: (Tick one) ☐ On the critical path ☐ Not on the critical path

Not on the critical path.

Left-shift: (Tick one) ☐ On the critical path ☐ Not on the critical path

Not on the critical path since it is shorter than the parallel operation of reading the register file.

Total latency = 400 (Inst-Mem) + 200 (Reg-File) + 120 (ALU) + 30 (MtoR) + 200 (Reg-File) = 950ps

Total latency: _____

